Ans1

Operating system is a system-level software layer that controls hardware and provides services to user applications. It is like a manager between user programs and machine resources. The first objective of OS is convenience, meaning users and programmers should not worry about hardware-level complexity. The second objective is efficient utilization of CPU, memory, and I/O so that throughput and responsiveness remain high.

It performs process scheduling, memory allocation, file and device handling, permission control, and system security. It also offers abstractions like files, virtual memory, and process APIs which make application development practical. Therefore, OS is both a control program and a resource allocator.

Ans2

A process is an independent executing program with separate address space and resources. If one process crashes, others are usually protected due to isolation. A thread is a smaller execution path inside a process and shares memory and resources of that process.

Threads are lightweight because creating/switching threads requires less overhead than processes. Also communication between threads is faster because they share address space directly. Example: in a browser, UI thread handles interaction, network thread fetches data, and rendering thread paints content, all in parallel under one process.

Ans3

CPU scheduling decides which process from ready queue should run next. It is necessary in multiprogramming because many processes are ready but only one can use CPU core at a time.

FCFS is non-preemptive and very simple. It follows arrival order, which is fair in queue sense but often inefficient. A long job at front delays smaller jobs and increases waiting time (convoy effect). Round Robin is preemptive and assigns each process a fixed quantum. This improves fairness and response time because no process can hold CPU too long. But RR has context-switch overhead, especially when quantum is too small. In practice, RR is preferred for interactive systems while FCFS may suit simple batch workloads.

Ans4

Deadlock is a condition in which processes are blocked forever because each is waiting for a resource held by another process. Coffman conditions explain when deadlock can occur:

1. Mutual exclusion: resource cannot be shared.
2. Hold and wait: process keeps one resource while asking another.
3. No preemption: resource cannot be forcefully taken.
4. Circular wait: chain of processes waiting for each other.

If all four happen together, deadlock is possible. Example: P1 holds printer and wants scanner; P2 holds scanner and wants printer. Neither proceeds. OS can handle deadlocks through prevention, avoidance, or detection/recovery methods.

Ans5

Virtual memory provides each process a large logical memory space, even when physical RAM is limited. The OS stores less active pages on disk and loads required pages into RAM on demand. Paging divides logical memory into pages and physical memory into equal-sized frames, then maps them via page table.

Main benefits are improved multiprogramming, process isolation, and reduced external fragmentation because contiguous physical allocation is not required. But if page faults happen too often, performance drops due to disk latency. So page replacement policy and frame allocation are important for efficiency.

Ans6

Synchronization is required when multiple threads/processes access shared data concurrently. Without synchronization, race conditions occur and outputs become non-deterministic. Critical section is the part of code where shared data is accessed and must be protected.

Mutex is a lock for mutual exclusion: one thread enters, others wait. Semaphore is a signaling counter and can allow one or more threads depending on binary or counting type. Semaphores are useful in producer-consumer control and resource pools. Proper synchronization ensures correctness, consistency, and safe concurrent execution.

Ans7

Fragmentation means wasted memory due to allocation patterns. Internal fragmentation happens when allocated block is larger than requested memory, leaving unused bytes inside that block. External fragmentation occurs when free memory is scattered into small holes and large contiguous allocation fails.

Internal fragmentation can be reduced by using better allocation sizes, while external fragmentation can be reduced using compaction or paging. Paging is especially effective against external fragmentation, though some internal waste may still exist in the last page.